

HW#4 Tianpu Zhao

(1)

The Verilog files are attached in the canvas submission. Here is a copy-paste version.
The .sdc file is not included in the submission (submission type only allows .v)

The module file: Q1_ALU.v

```
//=====
// 8-bit ALU (unsigned inputs), F[2:0] selects operation
// F=000: Q=a+b          (Cout = carry-out)
// F=001: Q=a-b          (Cout = borrow ; 1 if a<b, else 0)
// F=010: Q=a OR  b
// F=011: Q=a AND b
// F=100: Q=a XOR b
// F=101: Q=NOT a
// F=110: Q=arithmetic left shift a by 1 (<<1; LSB=0)
// F=111: Q=arithmetic right shift a by 1 (>>1; MSB holds sign bit a[7])
//=====
`timescale 1ns/1ps
module alu8 (
    input wire [7:0] a,
    input wire [7:0] b,
    input wire [2:0] F,
    output reg [7:0] Q,
    output reg      Cout // carry-out for add, borrow for sub
);
    // Internal 9-bit adders for proper carry/borrow handling
    wire [8:0] add9 = {1'b0, a} + {1'b0, b};
    wire [8:0] sub9 = {1'b0, a} + {1'b0, ~b} + 9'b000000001;

    always @* begin
        Q    = 8'h00;
        Cout = 1'b0;

        case (F)
            3'b000: begin // a + b
                Q    = add9[7:0];
                Cout = add9[8]; // true carry-out
            end
            3'b001: begin // a - b
                Q    = sub9[7:0];
            end
        endcase
    end
endmodule
```

```

        Cout = ~sub9[8]; // borrow = NOT(carry-out of a + ~b + 1)
    end
    3'b010: begin // a OR b
        Q    = a | b;
        Cout = 1'b0;
    end
    3'b011: begin // a AND b
        Q    = a & b;
        Cout = 1'b0;
    end
    3'b100: begin // a XOR b
        Q    = a ^ b;
        Cout = 1'b0;
    end
    3'b101: begin // NOT a
        Q    = ~a;
        Cout = 1'b0;
    end
    3'b110: begin // arithmetic left shift by 1 (same as logical for 2's comp)
        Q    = {a[6:0], 1'b0};
        Cout = 1'b0;
    end
    3'b111: begin // arithmetic right shift by 1 (preserve sign bit)
        Q    = {a[7], a[7:1]};
        Cout = 1'b0;
    end
    default: begin
        Q    = 8'h00;
        Cout = 1'b0;
    end
endcase
end
endmodule

```

The testbench: Q1_tb_ALU.v

```

`timescale 1ns/1ps

module tb_alu8;
    reg  [7:0] a, b;
    reg  [2:0] F;
    wire [7:0] Q;
    wire      Cout;

    alu8 dut(.a(a), .b(b), .F(F), .Q(Q), .Cout(Cout));

```

```

// Reference model for checks
function [8:0] add9(input [7:0] x, input [7:0] y);
    add9 = {1'b0,x} + {1'b0,y};
endfunction
function [8:0] sub9(input [7:0] x, input [7:0] y);
    sub9 = {1'b0,x} + {1'b0,~y} + 9'd1; // x - y
endfunction

task check(input [7:0] ta, input [7:0] tb, input [2:0] tF);
    reg [7:0] expected_Q;
    reg      expected_C;
    reg [8:0] tmp;
    begin
        a = ta; b = tb; F = tF;
        #1;

        case (tF)
            3'b000: begin
                tmp = add9(ta,tb);
                expected_Q = tmp[7:0];
                expected_C = tmp[8];
            end
            3'b001: begin
                tmp = sub9(ta,tb);
                expected_Q = tmp[7:0];
                expected_C = ~tmp[8];
            end
            3'b010: begin
                expected_Q = ta | tb;
                expected_C = 1'b0;
            end
            3'b011: begin
                expected_Q = ta & tb;
                expected_C = 1'b0;
            end
            3'b100: begin
                expected_Q = ta ^ tb;
                expected_C = 1'b0;
            end
            3'b101: begin
                expected_Q = ~ta;
                expected_C = 1'b0;
            end
            3'b110: begin
                expected_Q = {ta[6:0],1'b0};
                expected_C = 1'b0;
            end
            3'b111: begin

```

```

        expected_Q = {ta[7], ta[7:1]};
        expected_C = 1'b0;
    end
endcase

    if (Q!=expected_Q || Cout!=expected_C) begin
        $display("FAIL F=%03b a=%0d (%08b) b=%0d (%08b) -> Q=%0d (%08b)
Cout=%0b expected_Q=%0d (%08b) expected_C=%0b",
            tF, ta, ta, tb, tb, Q, Q, Cout, expected_Q, expected_Q, expected_C);
    end else begin
        $display("PASS F=%03b a=%0d (%08b) b=%0d (%08b) -> Q=%0d (%08b)
Cout=%0b",
            tF, ta, ta, tb, tb, Q, Q, Cout);
    end
end
endtask

initial begin
    $display("---- 8-bit ALU Test ----");
    // ADD tests (including 250+5 & 250+6)
    check(8'd12, 8'd34, 3'b000);
    check(8'd250, 8'd5, 3'b000); // expect Cout=0
    check(8'd250, 8'd6, 3'b000); // expect Cout=1

    // SUB tests (7-3 -> borrow 0; 3-7 -> borrow 1)
    check(8'd7, 8'd3, 3'b001);
    check(8'd3, 8'd7, 3'b001);

    // Logic ops
    check(8'b00101110, 8'b00011010, 3'b010); // OR
    check(8'b00101110, 8'b00011010, 3'b011); // AND
    check(8'b00101110, 8'b00011010, 3'b100); // XOR
    check(8'b00101110, 8'b00011010, 3'b101); // NOT

    // Shifts (show arithmetic right keeps MSB)
    check(8'b10110100, 8'b01010101, 3'b110); // <<1
    check(8'b10110100, 8'b01010101, 3'b111); // >>>1 (MSB stays 1)

    $display("All tests done.");
    #5 $finish;
end
endmodule

```

The sdc file (one-liner)

```
set_max_delay 1.0 -from [all_inputs] -to [all_outputs]
```

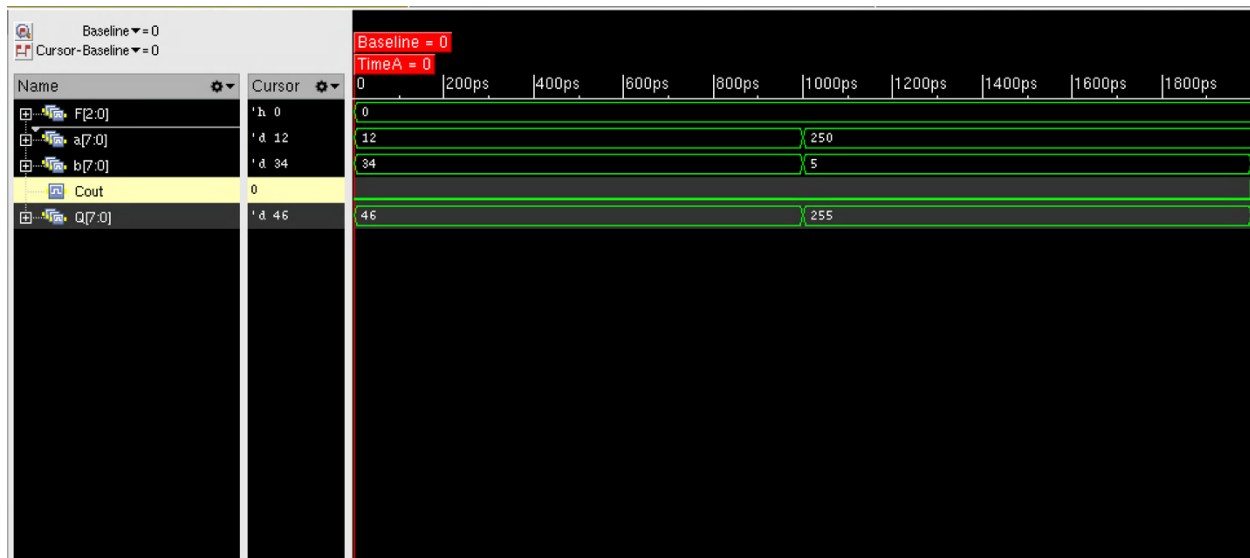
(2)

Waveforms:

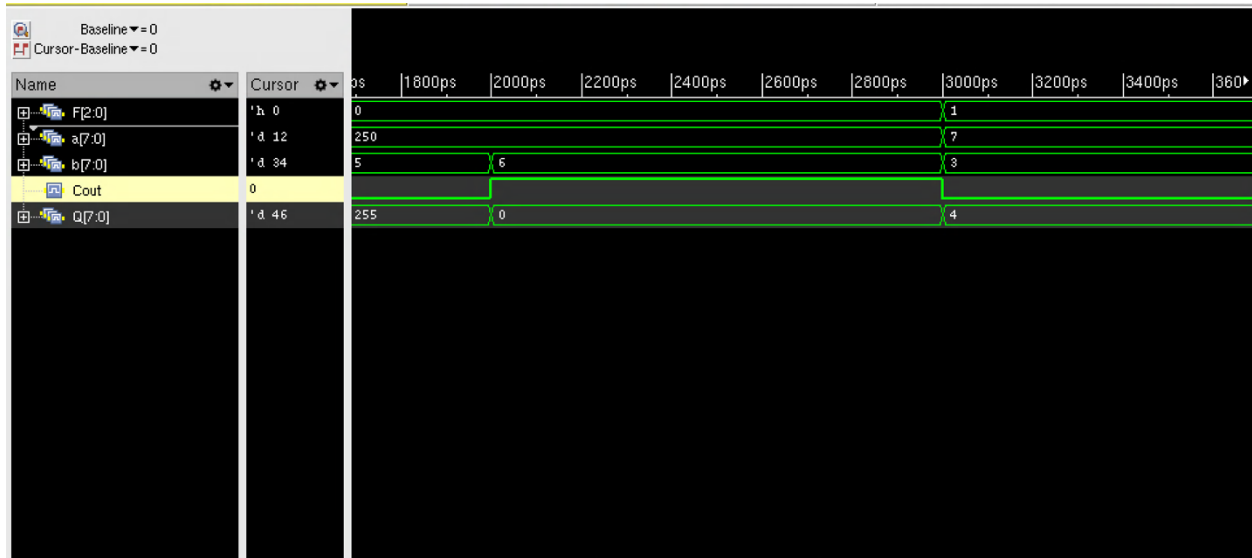
Included are examples of F = 000 (summation):

Case 1: a = 12, b = 34, Cout = 0, Q = 46

Case 2: a = 250, b = 5, Cout = 0, Q = 255



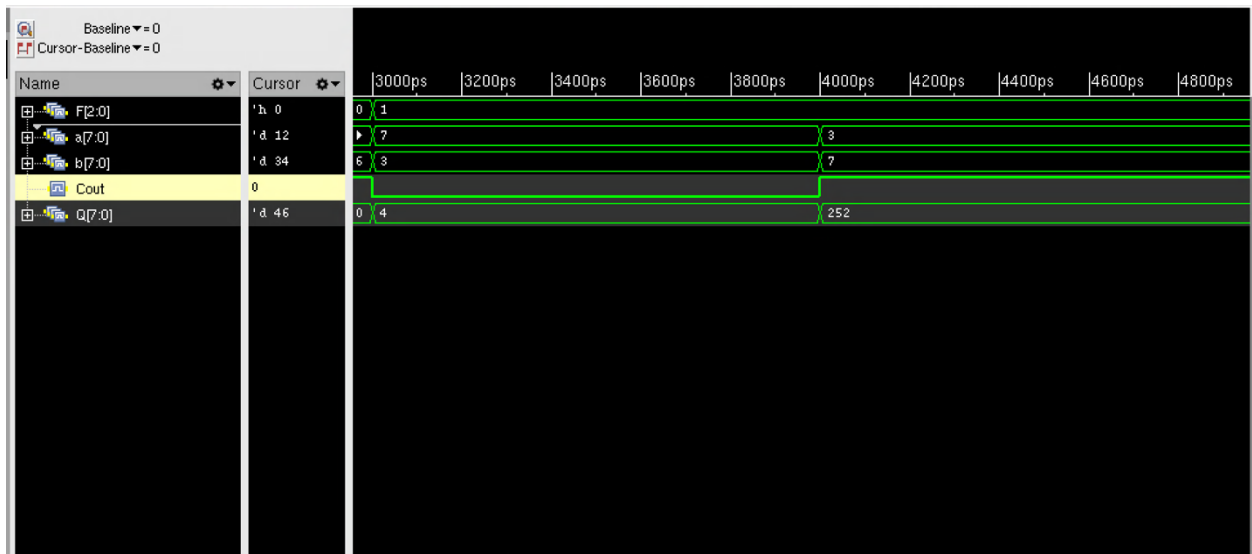
Case 3 (in the middle): $a = 250$, $b = 6$, $\text{Cout} = 1$, $Q = 0$



F = 001 (difference):

Left: $a = 7$, $b = 3$, $\text{Cout} = 0$, $Q = 4$

Right: $a = 3$, $b = 7$, $\text{Cout} = 1$, $Q = 252$



Left: F = 010 (a or b)

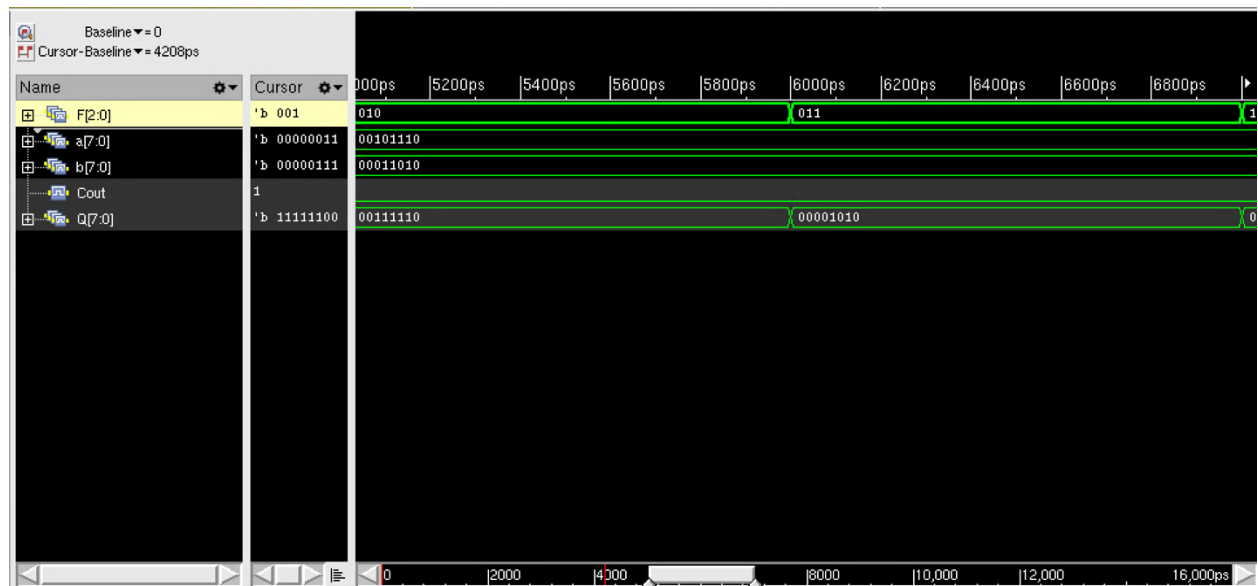
a=00101110, b=00011010

Q=00111110

Right: F = 011 (a and b)

Same set of a, b

Q = 00001010

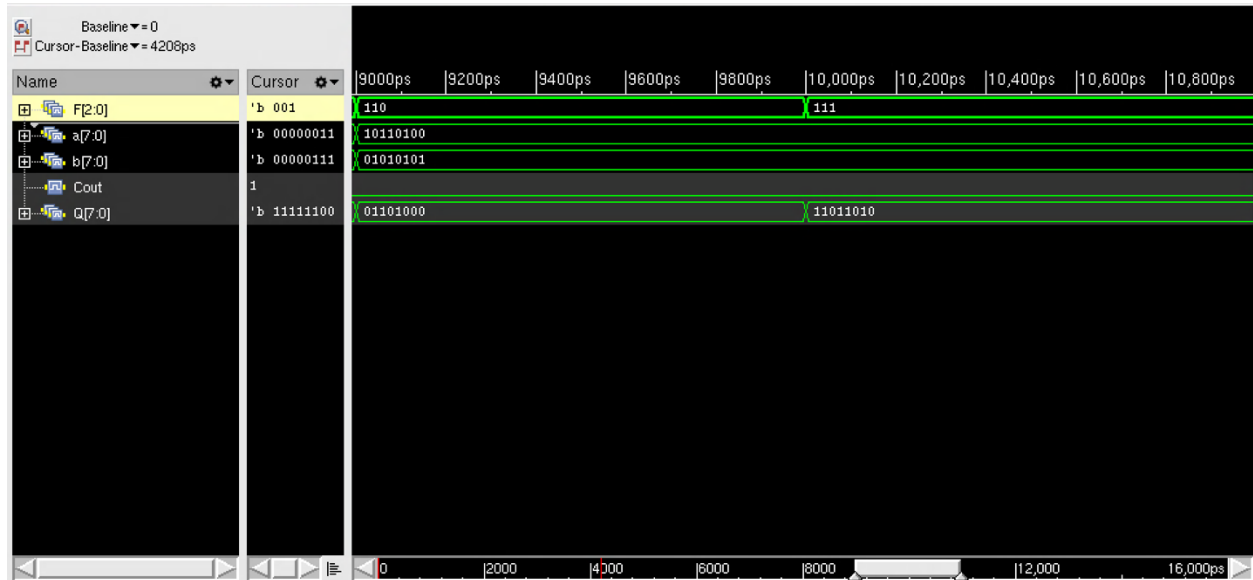


Left: F=110 (arithmetic left shift)

a=10110100 -> Q=01101000

Right: F=111 (arithmetic right shift)

a=10110100 -> Q=11011010

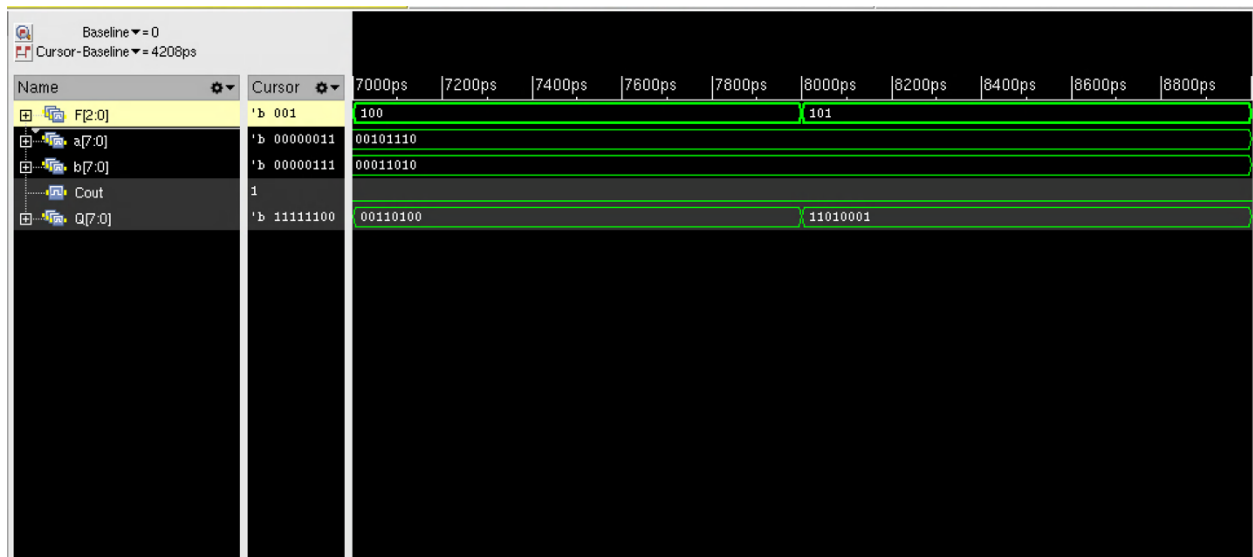


Left: F=100 a xor b

a=00101110, b=00011010 -> Q=00110100

Right: F=101 not a

a=00101110 -> 11010001



(3)

Total area = 396.673, cell count 160

Slack = 556ps

The area report area.rpt

=====

Generated by: Genus(TM) Synthesis Solution 18.14-s037_1

Generated on: Oct 30 2025 02:31:05 pm

Module: alu8

Operating conditions: typical

Interconnect mode: global

Area mode: physical library

=====

Instance	Module	Cell Count	Cell Area	Net Area	Total Area
----------	--------	------------	-----------	----------	------------

alu8		160	161.728	234.945	396.673
------	--	-----	---------	---------	---------

Timing report timing.rpt

=====

Generated by: Genus(TM) Synthesis Solution 18.14-s037_1

Generated on: Oct 30 2025 02:30:39 pm

Module: alu8

Operating conditions: typical

Interconnect mode: global

Area mode: physical library

=====

Path 1: MET (556 ps) Path Delay Check

Startpoint: (F) b[1]

Endpoint: (F) Q[7]

Capture Launch

Path Delay:+ 1000 -

Drv Adjust:+ 0 0

Arrival:= 1000

Required Time:= 1000

Data Path:- 444

Slack:= 556

Exceptions/Constraints:

max_delay 1000 constraints.sdc_line_1

#-----

Timing Point Flags Arc Edge Cell Fanout Load Trans Delay Arrival Instance

(fF) (ps) (ps) (ps) Location

#-----

b[1] - - F (arrival) 4 6.4 0 0 0 (-,-)

g5672__3772/ZN - A2->ZN R NOR2_X1 2 3.1 22 32 32 (-,-)

g5633__1840/ZN - A1->ZN R OR2_X1 1 1.2 6 26 57 (-,-)

g5614__2391/ZN - A1->ZN R AND2_X1 3 5.8 17 40 98 (-,-)

g5599__1857/ZN - B1->ZN F OAI21_X1 3 4.5 16 23 120 (-,-)

g5589__5266/ZN - B1->ZN R AOI21_X1 3 5.7 40 49 169 (-,-)

g5586/ZN - A->ZN F INV_X1 2 2.8 12 13 183 (-,-)

g5581__9682/ZN - B1->ZN R AOI21_X1 1 2.0 23 28 211 (-,-)

g5578/ZN - A->ZN F INV_X1 1 1.7 7 9 220 (-,-)

g5569__7344/ZN - B1->ZN R AOI21_X1 2 3.8 31 36 257 (-,-)

g5568/ZN - A->ZN F INV_X1 2 2.8 10 13 269 (-,-)

g5560__5953/ZN - B1->ZN R AOI21_X1 2 3.8 31 37 306 (-,-)

g5557/ZN - A->ZN F INV_X1 2 2.9 10 13 319 (-,-)

g5556__1309/ZN - A1->ZN F AND3_X1 1 1.8 7 31 350 (-,-)

g5551__1474/ZN - B->ZN R AOI211_X1 1 2.0 39 65 415 (-,-)

g5540__8780/ZN - B1->ZN F OAI221_X1 1 0.3 16 29 444 (-,-)

Q[7] <<< - F (port) - - - 0 444 (-,-)

#-----

(4)

Gate-level netlist file – see Q1_ALU_syn.v submitted

The waveform (all the results are shown in one figure)

